SERVING.DEMO release bundle — minimum gates before Iceberg

Treat each publish to `SERVING.DEMO` (and any Iceberg mirror) as a **release bundle**: **REFERENCE.CATALOG** seeds, **TRANSFORM / ANALYTICS** upstreams, thin `demo_*` views, **QA**, then the **replication job**. This runbook is the **minimal gate list** for **this repo** with **copy-paste** `dbt` **selectors**.

Related: `SERVING_DEMO_ICEBERG_TARGETS.md` (matrix + object intent), `SCHEMA_RULES.md` **§6** (PROD must not read `SERVING.DEMO`), `CONTRACT_RENT_AVM_VALUATION.md` (Cherre AVM snapshot / time truth).

---

0) Inventory (`models/serving/demo/`)

dbt model	Alias / note	Upstream (primary `ref`)
`demo_concept_rent_market_monthly`	thin slice	`concept_rent_market_monthly`
`demo_feature_rent_market_monthly`	FEATURE rent	`feature_rent_market_monthly_spine`
`demo_feature_listings_velocity_monthly`	listings FEATURE	`feature_listings_velocity_monthly_spi`
`demo_ref_product_type`	catalog	seed `product_type`
`demo_ref_metric`	catalog	seed `metric`
`demo_ref_bridge_product_type_metric`	catalog	seed `bridge_product_type_metric`
`demo_ref_concept`	catalog	seed `concept`
`demo_ref_metric_derived`	catalog	seed `metric_derived`
`demo_mart_county_ai_automation_risk`	row 83	`fact_county_ai_automation_risk`
`demo_model_county_ai_risk_dual_index`	row 82	`model_county_ai_risk_dual_index`
`demo_disposition_yield_property`	**disabled** unless `transform_dev_enable_disposition_yield_stack: true`	`concept_disposition_yield_property`
`demo_disposition_yield_portfolio`	same gate	`model_disposition_yield_portfolio`

---

1) Authoritative warehouse build (Snowflake-backed)

Use a **dev/stage target** whose `SERVING` / `REFERENCE` / `TRANSFORM` / `ANALYTICS` roles match where `SERVING.DEMO` and Iceberg land.

```
dbt parse
```

Catalog seeds (FK order matters on `reference` target — see `CATALOG_SEED_ORDER.md`):

```
# Quick path (typical dev):
dbt seed --select path:seeds/reference/catalog

# Or wave order from CATALOG_SEED_ORDER.md if you hit FK errors.
```

Build demo slice + all upstream parents (recommended single gate for “bundle compiles and runs”):

```
dbt run --select +path:models/serving/demo+
```

Integrated run + tests on the same subgraph (optional one-shot):

```
dbt build --select +path:models/serving/demo+
```

Targeted tests (explicit paths; use when you want a **fixed** surface without pulling every parent test):

```
dbt test --select "path:seeds/reference/catalog path:models/serving/demo path:models/transform/dev/concept
path:models/analytics/feature"
```

Stricter “everything that feeds demo” (includes tests defined on **any** upstream of `models/serving/demo`, e.g. Zillow facts, labor stack — slower, closer to a full bundle proof):

```
dbt test --select +path:models/serving/demo+
```

Gate: `dbt test` **green** for the scope you choose; disposition demos require `--vars 'transform_dev_enable_disposition_yield_stack: true'` (and upstream facts) if those nodes are enabled.

Feature leakage (CI or manual):

```
bash scripts/ci/check_feature_window_leakage.sh
```

Rent spine (when in bundle):

```
dbt test --select "feature_rent_market_monthly_spine,test_name:expression_is_true"
dbt test -s assert_feature_rent_output_schema_contract assert_synthetic_golden_feature_rent_keys_match
```

---

2) Schema / contract gates (not all are `dbt test`)

Gate	How to prove
§6 PROD isolation	Process + IAM + BI allowlist — **no PROD consumer** reads `SERVING.DEMO`. See `SCHEMA_RULES.md` §6 / matrix.
Object inventory	Every `SERVING.DEMO` relation you will replicate is **built** (`dbt run` above) and matches `SERVING_DEMO_ICEBERG_TARGETS.md` naming intent; `DESCRIBE TABLE` / `SELECT *` **LIMIT 1** matches Iceberg consumer contracts (catch rename drift).
Crosswalk QA	Enable `analytics_qa_serving_crosswalk_enabled: true`, build `path:models/serving/demo`, then `dbt run -s qa_serving_crosswalk_assertions` and materialize `QA_SERVING_CROSSWALK_ASSERTIONS`. Non-null keys + bridge resolution for `demo_ref_*` ↔ catalog vocabulary.

---

3) Upstream semantic correctness

Step	Command / action
Concept tests	`dbt test --select path:models/transform/dev/concept`
Feature tests	`dbt test --select path:models/analytics/feature` (extend with `+feature_rent_market_monthly_spine` if you need parent tests only)
Parity view	`dbt run -s qa_feature_concept_parity_diff` then review `QA_FEATURE_CONCEPT_PARITY_DIFF` — **resolve or document** drift before calling the lake canonical.
Cherre AVM snapshot	For Iceberg **v1**, **exclude** `demo_concept_*` slices that only expose misleading monthly ACF, **or** filter in a dedicated `demo_*` view — see `CONTRACT_RENT_AVM_VALUATION.md`.

---

4) Replication readiness (Snowflake SQL on exact `SERVING.DEMO` objects)

Run **after** `dbt run` (adjust database/schema if your target differs).

Row counts / `month_start` range (example pattern — repeat per `demo_*` table you replicate):

```
SELECT COUNT(*) AS n_rows, MIN(month_start) AS min_m, MAX(month_start) AS max_m
FROM SERVING.DEMO.DEMO_CONCEPT_RENT_MARKET_MONTHLY;
```

Distinct key cardinality (example for rent market grain):

```
SELECT
  COUNT(*) AS n_rows,
  COUNT(DISTINCT CONCAT(vendor_code, '|', geo_level_code, '|', geo_id, '|', TO_VARCHAR(month_start))) AS
n_distinct_keys
FROM SERVING.DEMO.DEMO_CONCEPT_RENT_MARKET_MONTHLY;
-- Expect n_rows = n_distinct_keys for a strict grain; investigate fan-out if not.
```

Joinability demo slice → `demo_ref_metric` (spot-check):

```
SELECT COUNT(*) AS orphan_metric_observe
FROM SERVING.DEMO.DEMO_CONCEPT_RENT_MARKET_MONTHLY AS c
LEFT JOIN SERVING.DEMO.DEMO_REF_METRIC AS m
```

```
  ON m.metric_id = c.metric_id_observe
WHERE c.metric_id_observe IS NOT NULL AND m.metric_id IS NULL;
```

Staleness:

```
SELECT MAX(month_start) AS max_m, DATEDIFF('month', MAX(month_start), DATE_TRUNC('month', CURRENT_DATE())) AS
months_behind
FROM SERVING.DEMO.DEMO_CONCEPT_RENT_MARKET_MONTHLY;
```

---

5) Iceberg-specific gates (operational)

Record in your **T4 / replication runbook** (outside or inside repo):

- **Target pattern** — Snowflake-managed Iceberg vs external catalog + object store (**one** pattern per environment).
- **Job semantics** — `INSERT OVERWRITE` vs incremental per table; `demo_ref_*` often **full refresh per release**.
- **Post-job** — rowcount ± tolerance, schema parity, partition filter smoke (`month_start`, `as_of`, etc.).

---

6) Release artifact (version pin)

For each publish, capture in ticket / run log:

Field	Example
Git SHA	`git rev-parse HEAD`
dbt target	`dev` / `staging`
Run timestamp	UTC wall clock
`demo_*` **list**	Names actually replicated
Catalog fingerprint	`shasum` of `seeds/reference/catalog/metric.csv` + `concept.csv` (or seed run id)

Minimum bar checklist (copy for PR / release)

1. `dbt parse` green
2. `dbt seed` for `path:seeds/reference/catalog` (or CATALOG_SEED_ORDER waves) green
3. `dbt run --select +path:models/serving/demo+` green
4. `dbt test` on chosen scope (**fixed paths** and/or `+path:models/serving/demo+`) green
5. `check_feature_window_leakage.sh` green (if FEATURE path in release)
6. **§6** — PROD isolation for `SERVING.DEMO` signed off
7. **Replication** dry run — rowcount / schema parity green
8. **Release note** — SHA + target + `demo_*` list + catalog fingerprint